CSL7110 — Assignment 4

Clustering and PageRank

Sandesh Suman
Roll Number: M25CSA034

GitHub: https://github.com/SandeshSuman123/CSL7110_Assignment4.git

---

Part 1: Clustering

Overview

In this part I implemented four functions for clustering using the UCI Spambase dataset which has 4601 points with 57 features (I excluded the 58th column which is the class label since we are doing unsupervised clustering). The two main algorithms are the Farthest First Traversal for k-center clustering and K-Means++.

Functions Implemented

readVectorsSeq(filename): This function reads the dataset file line by line, splits each line by comma, converts the values to floats, drops the last column (the spam/ham label), and returns a list of `pyspark.mllib.linalg.Vector` objects using `Vectors.dense()`.

kcenter(P, k) — Farthest First Traversal: The idea here is pretty simple. I pick the first point as the initial center (arbitrarily, index 0), then maintain an array `min_dists[i]` which stores the squared distance from point i to its nearest chosen center. At each step I pick the point with the maximum value in `min_dists` as the new center and update the array. This runs in $O(|P| \times k)$ since each of the k iterations does a single scan over all points. The algorithm gives a 2-approximation to the optimal k-center solution.

kmeansPP(P, k) — K-Means++ with Lloyd's Iterations: This has two phases. The first phase is the K-Means++ seeding where the first center is picked uniformly at random, and subsequent centers are chosen with probability proportional to D^2 (squared distance to nearest existing center). This gives an $O(\log k)$ approximation guarantee in expectation. The second phase runs Lloyd's iterations (assign each point to nearest center, recompute centers as means) for 20 iterations. Total complexity is still $O(|P| \times k)$ per iteration.

kmeansObj(P, C): Computes the average squared distance from each point to its nearest center. I used NumPy broadcasting to compute the full $(n \times k)$ distance matrix and then took the minimum per row.

Experiments

I ran three experiments with $k = 10$ and $k_1 = 50$:

1. **kcenter(P, k):** Just ran farthest first traversal and printed the time.
2. **kmeansPP(P, k) then kmeansObj:** Ran K-Means++ on the full dataset and computed the objective.

3. **Coreset approach — kcenter(P, k1) then kmeansPP(X, k):** First used kcenter to extract $k_1 = 50$ candidate centers as a coreset X, then ran kmeansPP on that smaller set to get the final k centers, and computed the objective on the full dataset.

The intuition for the third experiment is that a good coreset (large enough set of diverse points selected by farthest first) should capture the structure of the full dataset, so running K-Means++ on it should give good results without processing all 4601 points.

Analysis — Varying k

I varied $k \in \{2, 5, 10, 15, 20\}$ with $k_1 = 5 \times k$ and compared the K-Means++ objective on the full dataset versus the coreset approach. The results are shown in Figure 1.

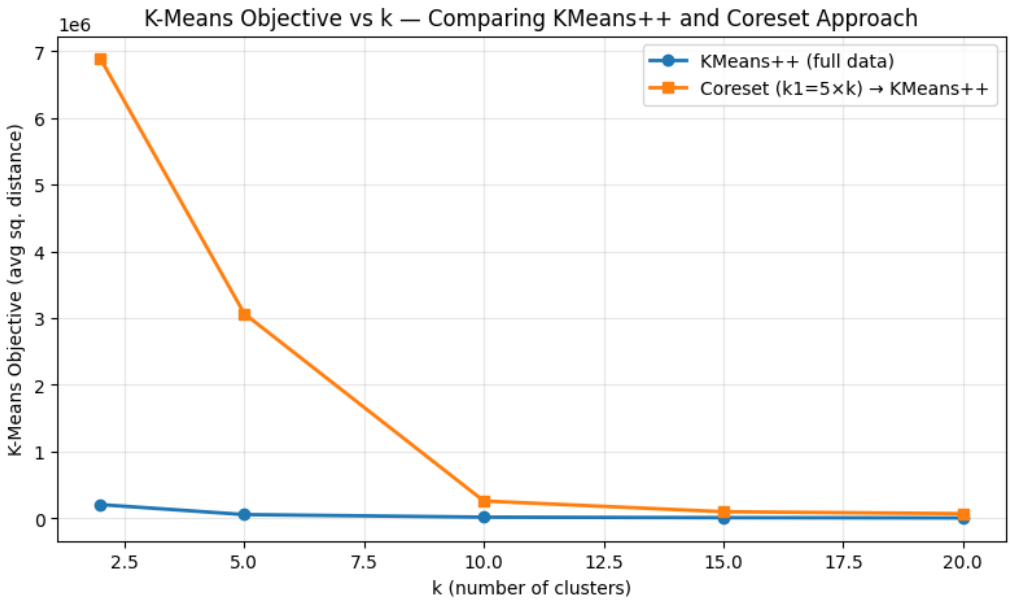

Figure 1: K-Means Objective vs k — comparing KMeans++ on full data and the coreset approach ($k_1 = 5k$).

From the plot it is clear that both methods improve as k increases. The coreset approach consistently gives a higher objective than running K-Means++ directly on the full dataset. This makes sense because the coreset is a lossy compression — it only has k_1 points — so running K-Means++ on it loses some fine-grained structure of the original data. However, as k gets larger the gap shrinks, which suggests that with a bigger coreset (proportionally larger k_1) we would get closer results. Overall the coreset idea is useful when the dataset is huge and you cannot afford to run K-Means++ on all points.

I also implemented a Spark RDD version of `kmeansObj` to parallelize the objective computation, which is important for very large datasets that do not fit in memory.

Part 2: Web Search — Inverted Index

Overview

In this part I built a complete inverted index for a collection of webpages and a search engine on top of it. The main idea is to map every word in the collection to a list of (page, position) tuples so that search queries can be answered efficiently.

Preprocessing

Before indexing, each word goes through the following steps:

- Convert to lowercase.
- Strip punctuation (replace with space).
- Skip stop words: {a, an, the, they, these, this, for, is, are, was, of, or, and, does, will, whose}.
- Normalize plural to singular using the given table (e.g., *structures* → *structure*).
- Note: stop words and punctuation are still counted for word position indices — they just aren't stored.

Classes Implemented

MySet: A custom set backed by a Python dictionary (keyed by `str(element)` for uniqueness). Supports `addElement`, `union`, and `intersection`.

Position: Stores a reference to a `PageEntry` and a 1-based word index. Getters: `getPageEntry()` and `getWordIndex()`.

WordEntry: Stores all positions across all documents where a word appears. Also computes term frequency (`getTermFrequency`) by counting positions in a given page and dividing by total word count.

MyHashTable: A hash table with chaining (capacity 1024). Uses Python's built-in `hash()` for the hash index. `addPositionsForWord` either inserts a new entry or merges with an existing one.

PageIndex: Per-page mapping from word to list of positions. Builds its index as the page file is read by `PageEntry`.

PageEntry: Reads the file for a given page name, tokenizes and preprocesses the text, and populates a `PageIndex`.

InvertedPageIndex: Global index across all pages. `addPage` adds all word entries from a page into the hash table. `getPagesWhichContainWord` looks up a word and collects all unique pages.

SearchEngine: Top-level class. `performAction` parses action strings:

- `addPage x` — reads and indexes the page.
- `queryFindPagesWhichContainWord x` — looks up pages containing word x.
- `queryFindPositionsOfWordInAPage x y` — returns positions of word x in page y.

TFIDF Scoring

For ranking, I implemented TFIDF. Given a word w and page p:

$$\text{relevance}_w(p) = \text{tf}_w(p) \times \text{idf}_w(p)$$

where $\text{tf}_w(p) = \frac{f_w(p)}{|W(p)|}$ (normalized frequency) and $\text{idf}_w(p) = \log \frac{N}{n_w}$ (N = total pages, n_w = pages containing w). As a demo, I computed TFIDF scores for the query word `push` across all six webpages and ranked them by relevance. Results are shown in Table 1.

Webpage	**TFIDF Score**
stack_datastructure_wiki	0.011916
stack_oracle	0.007683
stack_cprogramming	0.003544
stackoverflow	0.000000
stacklighting	0.000000
stackmagazine	0.000000

Table 1: TFIDF scores for query word `push` across the six webpages. Pages with a score of 0.0 do not contain the word `push` at all. The data structures pages rank highest as expected since `push` is a core stack operation.

Validation

I ran the provided `actions.txt` and compared outputs against `answers.txt` line by line. The implementation correctly handles all the edge cases mentioned in the spec including the "No webpage contains word x" and "No webpage y found" messages.

Part 3: PageRank on Spark

Overview

In this part I implemented the iterative PageRank algorithm on Spark using RDDs. The graph has $n = 1000$ nodes and $m = 8192$ directed edges (after deduplication). The dataset also has a smaller graph `small.txt` with 53 nodes for validation.

Algorithm

The PageRank vector r is defined by:

$$r = \frac{1-\beta}{n}\mathbf{1} + \beta M r$$

where $M_{ij} = \frac{1}{\deg(i)}$ if edge $(i \to j) \in E$ and 0 otherwise, and $\beta = 0.8$ is the damping factor. The teleport probability is $1 - \beta = 0.2$.

The iterative update from step $i-1$ to i is:

$$r^{(i)} = \frac{1-\beta}{n}\mathbf{1} + \beta M r^{(i-1)}$$

Initialized with $r^{(0)} = \frac{1}{n}\mathbf{1}$.

Spark Implementation

I represented the graph as an RDD of adjacency lists (`links_rdd`: `(node, [neighbors])`). At each iteration:

1. Join `links_rdd` with the current `ranks_rdd` to get `(node, ([neighbors], rank))`.
2. FlatMap to emit `(neighbor, rank / out_degree)` for each neighbor.
3. ReduceByKey to sum contributions per node.
4. MapValues to apply teleportation: `rank = teleport + beta * sum`.

The `links_rdd` is cached since it is used in every iteration. Duplicate edges are removed with `distinct()` before building the adjacency list.

Validation on Small Graph

I first ran 40 iterations with $\beta = 0.8$ on `small.txt` (53 nodes). The top score came out to approximately 0.036, matching the expected value given in the assignment. This gave confidence that the implementation is correct.

Results on Full Graph

Running 40 iterations with $\beta = 0.8$ on `whole.txt` (1000 nodes):

The top 5 and bottom 5 nodes by PageRank score are reported in Tables 2 and 3 below.

Rank	Node ID	PageRank Score
1	263	0.002020292911815182184
2	537	0.001943341571453149700
3	965	0.001925447807166263100
4	243	0.001852634016241731000
5	285	0.001827373721700645144

Table 2: Top 5 nodes with the highest PageRank scores on `whole.txt` ($\beta = 0.8$, 40 iterations). These nodes receive the most link authority from other high-ranked nodes in the graph.

Rank	Node ID	PageRank Score
996	408	0.000038779848719291705
997	424	0.000035481538649301454
998	62	0.000035314810510596274
999	93	0.000035135689375165774
1000	558	0.000032860185252152970

Table 3: Bottom 5 nodes with the lowest PageRank scores on `whole.txt`. These nodes have few or weak inbound links and thus accumulate very little rank from the graph structure.

Convergence Analysis

I also tracked the L1 norm of the rank change at each iteration ($\|r^{(i)} - r^{(i-1)}\|_1$) to analyze convergence. The result is shown in Figure 2.

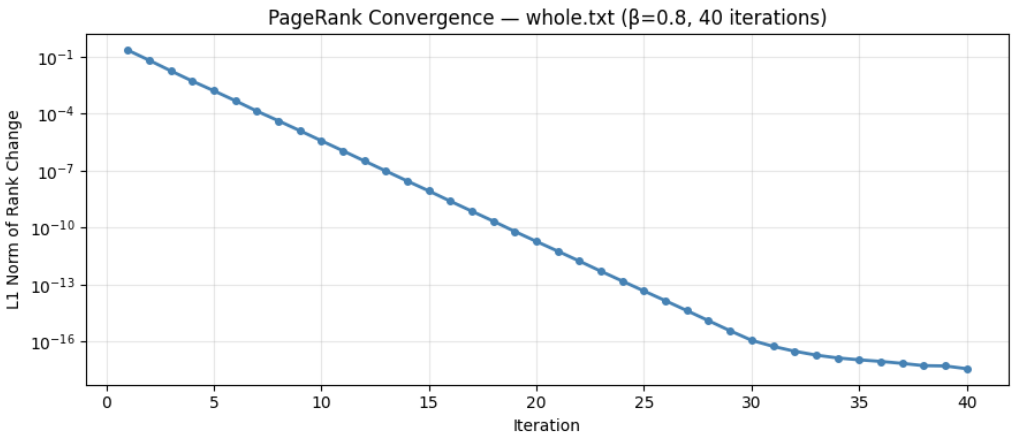

Figure 2: PageRank convergence on `whole.txt` ($\beta = 0.8$, 40 iterations). The L1 norm of rank change drops steadily on a log scale, confirming that the algorithm converges.

The convergence is smooth and nearly geometric on the log scale, which is expected — the spectral gap of the PageRank transition matrix controls the rate of convergence and is related to β. By around iteration 30 the L1 norm is already below 10^{-15}, which is essentially machine precision, so 40 iterations is more than enough for this graph.